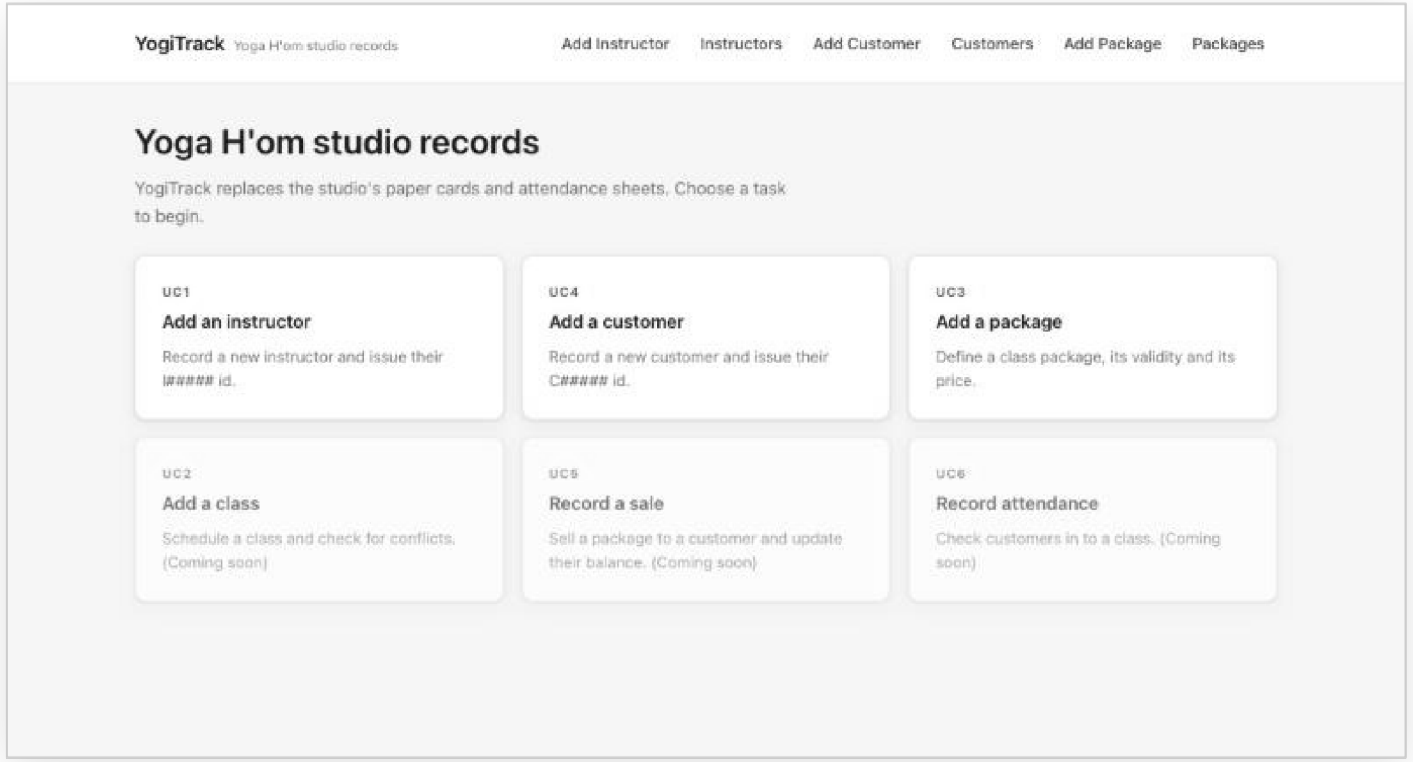
YogiTrack

A MERN application replacing paper record-keeping at the Yoga H'om studio. Application 1.0: three use cases, deployed and live.

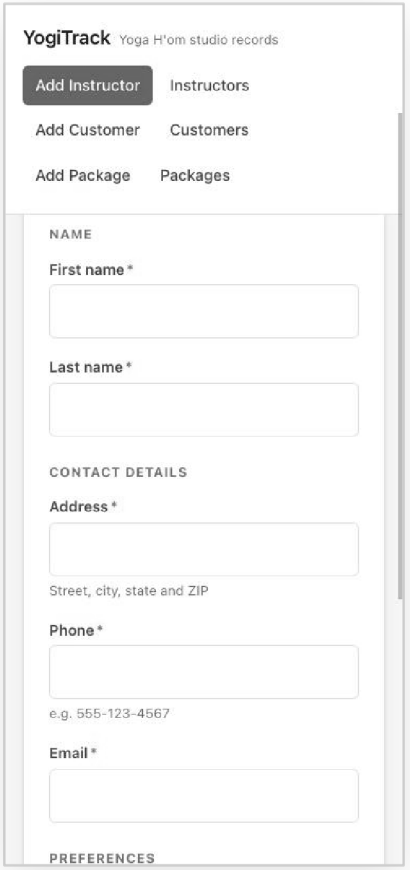
Zach Blitz · University of Oklahoma · Fall 2026

Live: yogitrack-zb-9cc3ff17df1c.herokuapp.com · Repo: github.com/ZB56/yogitrack

Live on Heroku, backed by MongoDB Atlas

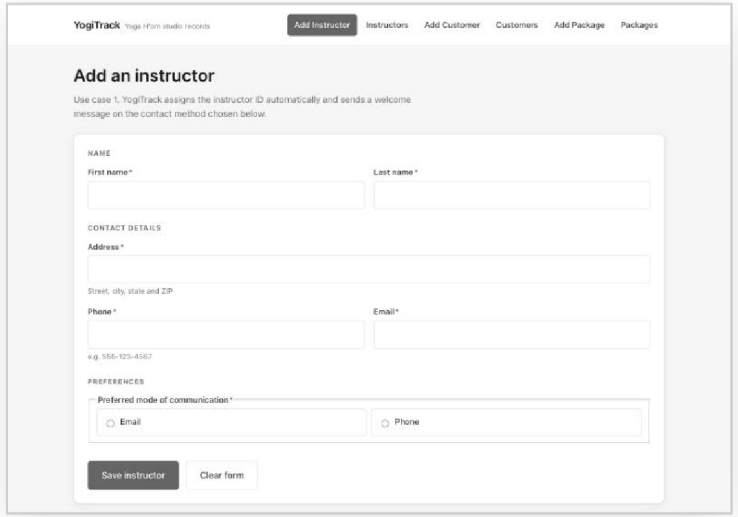


Home. One tile per use case; the Part 2 tiles are present but greyed out, so the whole system's scope and the Part 1 increment are both visible at a glance.

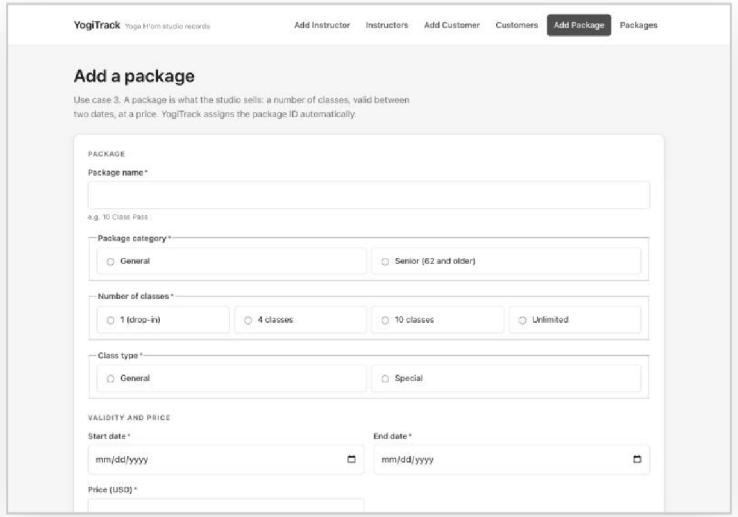


The same app at 375px. The nav wraps, forms collapse to one column, tables scroll horizontally.

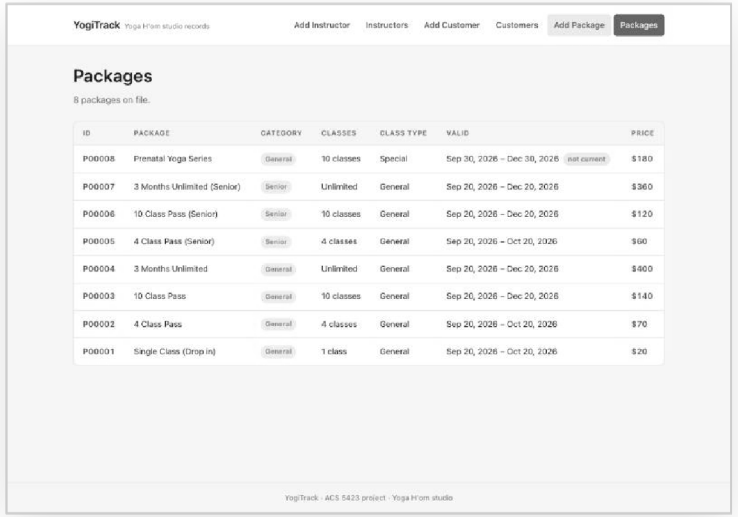
The Part 1 screens



UC1 · Add instructor. UC4 Add customer is the same component with different wording and an opening balance of 0.



UC3 · Add package. "Unlimited" is a first-class radio option, not a special number.



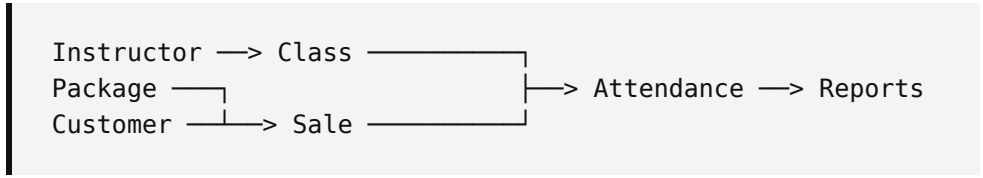
Packages list. Reads like the studio's printed price list, senior rates included. Instructor and customer lists follow the same pattern.

Three of seven implemented; the three everything else depends on

#	USE CASE	ACTOR	STATUS
UC1	Add an instructor	Manager	COMPLETE
UC4	Add a customer	Manager	COMPLETE
UC3	Add a package	Manager	COMPLETE
UC2	Add a class	Manager	PART 2
UC5	Record a sale	Manager	PART 2
UC6	Record class attendance	Instructor	PART 2
UC7	Generate studio reports	Manager	PART 2

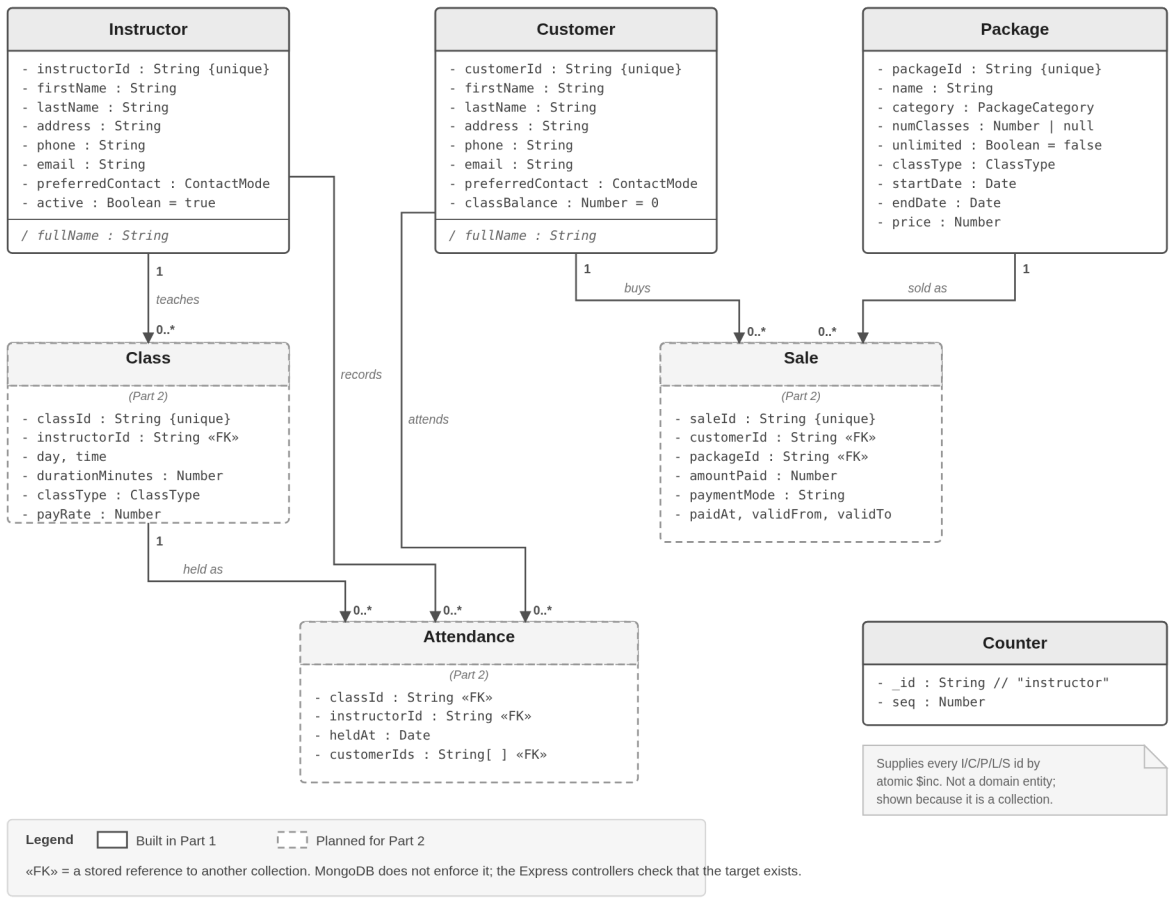
The course spec labels two different use cases "5"; renumbered 1–7 in the report and code.

WHY THESE THREE FIRST



- No class without an instructor; no sale without a customer and a package.
- Building the roots means **Part 2 is purely additive**: nothing from Part 1 gets redesigned.
- UC1 and UC4 share ~90% of their logic, so the second was built by **extracting** what they share, not copying it.

Six collections plus a counter, with rules enforced in the schema



Readable IDs from an atomic counter

`findOneAndUpdate` + `$inc` on a `counters` document → I00001, C00001, P00001. Counting documents would race; `$inc` can't.

MongoDB does not enforce references

No foreign keys, no JOIN. Every cross-collection reference is checked in the Express controller. The «FK» marks are intent, not a constraint.

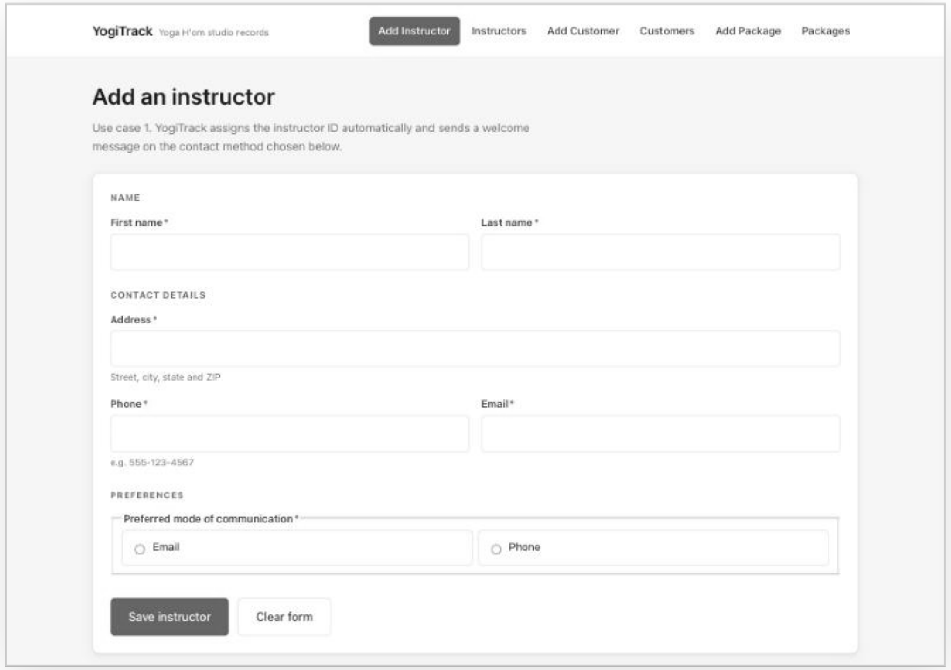
"Unlimited" is a flag, not a magic number

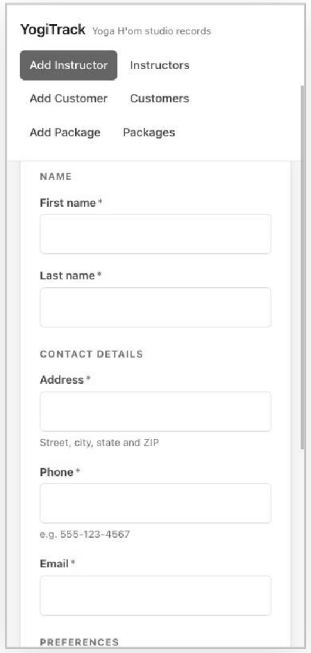
`unlimited: true`, `numClasses: null`. The `required` rule is a function that reads the document, so the count is mandatory only for counted packages.

Validate first, then allocate the ID

Otherwise every rejected form consumes a number and the studio sees gaps that look like lost records.

Mobile first, one form component, honest state





- **Responsive.** Base CSS is the phone layout; media queries widen at 640px and 900px. A manager at the front desk with a phone is a real user.
- **One `PersonForm`, used twice.** UC1 and UC4 behave identically because they *are* the same screen, configured by props.
- **Errors beside the field**, cleared as soon as it is edited. Server validation errors map to the same spots.
- **Never looks idle while working.** Buttons disable and relabel: "Checking name…", then "Saving…".
- **Accessible by default.** Labels bound by `id`, `aria-describedby` on errors, fieldset/legend grouping, 44px touch targets, visible focus.
- **Hand-written CSS**, no framework: every rule is one that can be explained.

Applied from Weeks 1–4, and what Part 1 needed before the course got there

FROM THE COURSE, APPLIED HERE

- **Week 1, patterns and protocols.** MERN as the stack; a three-tier client-server design (React → Express → MongoDB); HTTP status codes as the API contract: 201, 400, 404, 409.
- **Week 2, setting up the stack.** Node.js and Express handling requests and responses; RESTful routes under `/api`; MongoDB as the database.
- **Week 3, HTML and CSS.** Forms and form widgets (radio groups, `fieldset`/`legend`); semantic markup and accessibility; selectors and the box model.
- **Week 4, CSS and JavaScript.** Flexbox and Grid for the responsive layout; form styling; JS functions, objects, arrays, `Date`, exception handling. Discussion 2: MongoDB setup, Atlas.

BEYOND WEEKS 1–4

- **React** is Week 7. Function components, `useState`/`useEffect`, controlled inputs and React Router were self-taught for Part 1, with **Vite** as the build tool.
- **Asynchronous JavaScript.** Promises, `async`/`await`, `fetch`, and ES modules go past the Week 4 fundamentals, and every database call depends on them.
- **Mongoose** as the ODM: schemas with validation, virtuals, a compound index, function-valued `required`, atomic `$inc` with `upsert`.
- **Express 5**: errors thrown in async handlers reach one error middleware automatically; routes → controllers → services → models layering.
- **GitHub Actions CI** that boots the server against a MongoDB service container and hits the endpoints; Heroku waits for it before deploying.

What broke, and what each break taught

Heroku build failed:
`vite: not found`

Heroku sets `NODE_ENV=production`, and npm then skips devDependencies, so the bundler was never installed. Fixed with `npm ci --include=dev` for the client rather than misfiling Vite as a runtime dependency. Reproduced locally before trusting the fix.

Built, then crashed on boot: Atlas authentication failed

The app connects to the database *before* it listens and exits with a clear message if it can't. That crash was the correct outcome; the alternative is a running app that fails on every request. Also caught a connection string missing its database name, which would have silently written to `test`.

ID numbers had gaps

Found by a test, not by design: generating the ID before validation burned a counter value on every rejected form. Reordered to validate first.

Asynchrony, coming from Java

Nearly every database call returns a Promise. A forgotten `await` doesn't fail; it silently hands you the Promise instead of the value. This was the single most frequent source of confusion.

`fetch`
does not reject on a 4xx

A 400 is a successful round trip as far as `fetch` is concerned. The API layer checks status explicitly and converts it into a typed error carrying per-field messages.

A rule that lives only in the UI is not a rule

The duplicate-name check, field validation and the "balance starts at 0" rule are all enforced server-side too, because a request can always arrive without the form.

A report that lists only passes isn't useful

The verification section records the defect it found, not just the 36 cases that passed.

Production app on Heroku, backed by MongoDB Atlas

01 **Home.** One tile per use case; Part 2 tiles visibly greyed out so the increment is obvious.

02 **Add instructor, empty submit.** Seven field-level errors, no request sent.

03 **Add instructor, complete.** Generated ID (I000xx) and the welcome message routed by the chosen contact method.

04 **Same name again, different case.** A warning with "Yes, add anyway" / "Cancel and edit", not an error. Server answers 409 with `requiresConfirmation`.

05 **Instructors list.** Newest first, contact method as a badge.

06 **Add customer.** The same form; success panel shows opening balance 0.

07 **Add package: 3-month unlimited.** "Unlimited" is a first-class radio option; the list reads like the studio's printed price list.

08 **Pipeline.** GitHub Actions run passing; Heroku deploy that waited on it.

3
USE CASES, END TO END

12
API ENDPOINTS UNDER /API

7
SCREENS, EACH DOING ONE THING

36
VERIFICATION CASES RUN, API AND UI

14
COMMITS, EACH A WORKING INCREMENT

Part 2 definition of done, and what is left to reach it

DELIVERABLES DUE OCT 16 (FRIDAY, WEEK 8)

REQUIRED	STATUS
Application deployed on the cloud, code repo on GitHub	DONE · PART 1
CI/CD pipeline deploying incremental changes	DONE · PART 1
Updated project report, submitted as PDF	UPDATE

Deploying to Heroku in Part 1 instead of zyLabs means the first two are already met; every Part 2 commit ships through the pipeline that exists today.

OPEN DECISIONS THAT GATE THE WORK

How long is a class? Needed for the conflict check.	UC2
Manager vs. Instructor: role selector or real login?	UC6
Diagram's five reports or the written spec's four?	UC7

WHAT "ALL USE CASES FULLY IMPLEMENTED" STILL REQUIRES

01 **UC2 Add class.** Schedule-conflict detection with suggested alternatives; confirmation to manager and instructor.

02 **UC5 Record sale.** Amount validated against the package rate, dates against package rules; class balance updated and shown.

03 **UC6 Record attendance.** Instructor's class list, date/schedule mismatch warning, negative-balance override, check-in message per customer.

04 **UC7 Studio reports.** Package sales, instructors with classes and check-ins, customers with package status, monthly teacher payment. Aggregation pipelines, the "creative use of the stack" the rubric asks for.

05 **Real message delivery** behind the existing messaging interface, so UC1, UC4 and UC6 actually send.

06 **Report update.** Four new use cases, revised UML, decisions 9–12 resolved, Part 2 verification cases.